

Modern Dotfiles Management

Cross-Platform Development Environment with **Nix**

Press Space for next page →



What are Dotfiles?

Dotfiles are **configuration files** that customize your development environment

- 🏠 **Personal Setup:** Shell configurations, editor settings, aliases
- 🛠️ **Tool Configuration:** Git, terminal, development tools
- 🎨 **Customization:** Themes, shortcuts, productivity enhancements
- 🚀 **Automation:** Scripts, workflows, environment setup
- 📦 **Package Management:** Software installation and updates

The Challenge

- Multiple machines (work, personal, servers)
- Different operating systems (macOS, Linux, WSL)
- Keeping configurations in sync
- Reproducible development environments

Why Nix for Dotfiles?

Nix provides **declarative**, **reproducible**, and **reliable** system management

Traditional Approach

```
1  # Install packages manually
2  brew install git neovim
3  apt install git neovim
4
5  # Configuration scattered
6  ~/.gitconfig
7  ~/.vimrc
8  ~/.zshrc
9  ~/.config/...
10
11 # Manual synchronization
12 # Version conflicts
13 # Platform differences
```

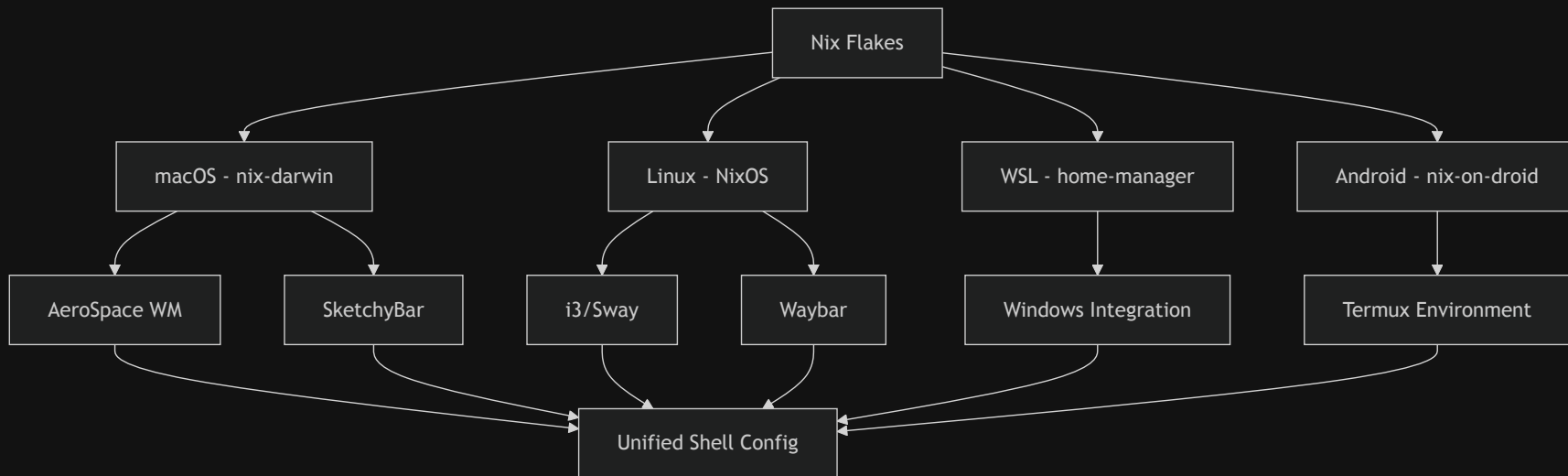
Nix Approach

```
1  {
2    programs.git.enable = true;
3    programs.neovim.enable = true;
4
5    home.packages = with pkgs; [
6      git neovim tmux
7    ];
8  }
```

- **Declarative:** Describe what you want
- **Reproducible:** Same result every time
- **Atomic:** All-or-nothing updates
- **Rollback:** Easy to undo changes

Our Multi-Platform Architecture

Supporting **4 platforms** with unified configuration



Project Structure Overview

```
1  dotfiles/
2  └─ nix/platforms/
3      └─ flake.nix          # Main entry point
4      └─ common/
5          └─ platform-detection.nix # Auto-detect platform
6          └─ packages/core.nix     # Essential tools
7          └─ themes/colors.nix     # Unified theming
8          └─ development/default.nix # Dev environment
9      └─ darwin/                # macOS specific
10     └─ linux/                  # Linux specific
11     └─ wsl/                    # WSL integration
12     └─ android/                # Termux support
13 └─ configs/
14     └─ shell/                  # Zsh, aliases, functions
15     └─ terminal/wezterm.lua    # Terminal config
16     └─ wm/aerospace/           # Window manager
17     └─ editors/neovim/        # Editor setup
18 └─ scripts/                   # Automation tools
```

Key Features & Capabilities

Development Environment

- **LSP Integration:** Language servers for 15+ languages
- **AI Tools:** GitHub Copilot, ChatGPT integration
- **Container Support:** Docker, Podman, Kubernetes
- **Modern CLI Tools:** ripgrep, fd, bat, exa, fzf

Security & Secrets

- **SOPS-nix:** Encrypted secrets management
- **Git-crypt:** Selective file encryption
- **SSH Key Management:** Automated key generation
- **Secure Defaults:** Hardened configurations

Automation & CI/CD

- **GitHub Actions:** Multi-platform testing
- **Infrastructure as Code:** Terraform, Ansible
- **Health Monitoring:** System diagnostics
- **Auto-deployment:** Production pipelines

Monitoring & Analytics

- **Performance Metrics:** Resource monitoring
- **Usage Analytics:** Tool usage patterns
- **Health Checks:** System validation
- **Reporting:** Automated status reports

Live Demo: Package Management

Real-time package installation with Nix

Traditional Package Management

```
1  # Different commands per platform
2  brew install htop          # macOS
3  apt install htop           # Ubuntu
4  pacman -S htop             # Arch
5  choco install htop         # Windows
6
7  # Version conflicts
8  # Dependency hell
9  # Manual updates
10 # No reproducibility
```

Nix Declarative Approach

```
1  # Single configuration
2  home.packages = with pkgs; [
3    htop
4    neovim
5    git
6    docker
7  ];
8
9  # ✅ Same version everywhere
10 # ✅ Automatic dependencies
11 # ✅ Atomic updates
12 # ✅ Easy rollbacks
```

Quick Installation Commands

```
1  # Install on any platform
2  nix run .#setup
3
4  # Update everything
5  nix flake update && nix run .#rebuild
```

Advanced Features Showcase



AI-Powered Development

```
1  {
2    dotfiles.development = {
3      enable = true;
4      profile = "ai-powered";
5
6      ai.tools = [
7        "github-copilot"
8        "chatgpt-cli"
9        "aider-chat"
10     ];
11
12     languages = [
13       "python" "rust" "go"
14       "typescript" "nix"
15     ];
16   };
17 }
```



Enterprise Security

```
1  {
2    dotfiles.security = {
3      enterprise.enable = true;
4      securityLevel = "high";
5
6      encryption = {
7        sops = true;
8        git-crypt = true;
9      };
10
11     monitoring = {
12       auditLogs = true;
13       compliance = "soc2";
14     };
15   };
16 }
```



Universal Platform Support

Real-World Benefits



Productivity Gains

- **Setup Time:** New machine ready in 15 minutes
- **Consistency:** Identical environment across all devices
- **No Configuration Drift:** Declarative prevents divergence



Security Improvements

- **Encrypted Secrets:** SOPS + age encryption
- **Audit Trail:** All changes tracked in Git
- **Compliance Ready:** SOC2 framework support



Development Velocity

- **Instant Dev Environments:** `nix develop`
- **Language Support:** 15+ languages with LSP

Getting Started

Quick Setup (Any Platform)

```
1 # 1. Clone the repository
2 git clone https://github.com/gapul/dotfiles.git
3 cd dotfiles
4
5 # 2. Run automated setup
6 nix run .#setup
7
8 # 3. Enjoy your new environment! 🚀
```

Common Commands

```
1 nix run .#rebuild           # Apply configuration changes
2 nix run .#health           # System health check
3 nix run .#analyze           # System analysis
4 nix flake update            # Update all packages
```



[Full Documentation](#) |



[Live Demo](#)

Thank You!

Modern Dotfiles Management with Nix

Questions & Discussion 

